FIAP – TURMA 1TDSPY

FERNANDO OLIVEIRA FRANCELINO SARDINHA – RM570196

JONATAN VIEIRA FEITOSA – RM570452

MARCELO CANDIDO DA MATA JUNIOR – RM569584

VINICIUS LIBERATO DOS ANJOS – RM571480

DAVI FELIX CUNHA – RM569393

# SOULGAME – DOCUMENTAÇÃO TÉCNICA

CHALLENGE SPRINT 4

São Paulo

2026

FERNANDO OLIVEIRA FRANCELINO SARDINHA – RM570196

JONATAN VIEIRA FEITOSA – RM570452

MARCELO CANDIDO DA MATA JUNIOR – RM569584

VINICIUS LIBERATO DOS ANJOS – RM571480

DAVI FELIX CUNHA – RM569393

# SOULGAME – DOCUMENTAÇÃO TÉCNICA

## CHALLENGE SPRINT 4

Trabalho de avaliação semestral apresentado à FIAP como requisito parcial à aprovação na disciplina Domain Driven Design Using JAVA.

Prof. Rodrigo Vieira Pinto

Turma 1TDSPY

São Paulo

2026

## 1. ESCOPO E OBJETIVO

O **SoulGame** é uma plataforma digital de gamificação da sustentabilidade, desenvolvida para incentivar a adoção de hábitos ambientalmente responsáveis por meio de desafios, pontuações, conquistas e competição saudável entre os participantes.

A proposta parte de um problema cotidiano: embora muitas pessoas reconheçam a importância da preservação ambiental, nem sempre conseguem transformar esse conhecimento em atitudes práticas e constantes. O SoulGame busca reduzir essa distância entre conscientização e ação, utilizando elementos característicos dos jogos para tornar a participação em atividades sustentáveis mais interessante, acessível e motivadora.

Em vez de apresentar a sustentabilidade apenas como uma obrigação individual, a plataforma transforma ações positivas em oportunidades de evolução dentro de um ambiente interativo. Assim, atividades como economizar recursos, reduzir a geração de resíduos e participar de iniciativas ambientais podem ser associadas a desafios que incentivam o engajamento contínuo.

O projeto também se relaciona aos **Objetivos de Desenvolvimento Sustentável (ODS)** da Organização das Nações Unidas, permitindo que as ações cadastradas sejam vinculadas a diferentes temas da Agenda 2030.

## 2. O SOULGAME – PRINCIPAIS FUNCIONALIDADES DA SOLUÇÃO

O funcionamento do SoulGame é baseado em um ciclo de participação, realização de ações, registro de evidências e acompanhamento da evolução do usuário.

### 2.1. Cadastro e acesso à plataforma

O participante cria sua conta informando seus dados cadastrais e passa a ter acesso ao ambiente do jogo. Cada usuário possui um perfil individual, no qual são armazenadas informações relacionadas à sua participação, pontuação e evolução.

### 2.2. Escolha de ações sustentáveis

Após acessar a plataforma, o jogador pode consultar as atividades sustentáveis disponíveis. Cada ação possui uma descrição, um objetivo ambiental, um nível de dificuldade e uma pontuação-base.

As ações podem estar relacionadas a um dos 17 Objetivos de Desenvolvimento Sustentável, permitindo que o participante compreenda a finalidade ambiental da atividade escolhida.

### 2.3. Realização dos desafios

O jogador seleciona uma ação e procura realizá-la no mundo real. A plataforma também permite agendar desafios, estabelecendo uma data para sua realização. Essa funcionalidade incentiva o planejamento e a criação de uma rotina de participação, em vez de limitar o jogo a atividades isoladas.

### 2.4. Registro da atividade e envio de evidências

Após realizar uma ação, o participante pode registrar sua atividade na plataforma e enviar uma imagem como evidência. O sistema recebe os dados do registro e da evidência, associando-os ao usuário e à ação sustentável correspondente.

Os registros possuem estados de validação, como **PENDENTE, VALIDADO e REJEITADO**, permitindo distinguir as atividades submetidas daquelas efetivamente aprovadas. A interface também contempla situações em que a análise automatizada da evidência não é conclusiva, mantendo a atividade pendente de revisão. Dessa maneira, o envio de uma imagem não representa, por si só, a aprovação definitiva da atividade.

### 2.5. Pontuação e evolução

O SoulGame utiliza um sistema de pontos para representar o progresso dos participantes. As ações possuem pontuação-base, e o modelo de usuários contempla informações como total de pontos, nível e sequência de participação (*streak*).

Esses elementos permitem acompanhar a evolução individual e incentivar a continuidade das atividades sustentáveis. A pontuação é uma recompensa simbólica dentro do jogo, não um valor financeiro.

### 2.6. Ranking e competição saudável

Os participantes podem acompanhar sua posição em um ranking organizado pela pontuação acumulada. O ranking introduz um elemento competitivo à experiência, estimulando os jogadores a realizar novas atividades e melhorar seu desempenho.

O objetivo não é apenas premiar quem possui mais pontos, mas incentivar a participação frequente e ampliar o envolvimento coletivo com a sustentabilidade.

### 2.7. Conquistas e reconhecimento

O jogo também possui um sistema de conquistas, que funciona como uma forma de reconhecimento pela evolução dos participantes. As conquistas possuem critérios cadastrados e podem estar relacionadas a determinadas ações ou aos ODS. A plataforma permite consultar as conquistas associadas a cada usuário, distinguindo aquelas desbloqueadas das demais. Esse mecanismo acrescenta objetivos de progressão além da pontuação e do ranking.

### 2.8. Histórico e acompanhamento

O participante pode consultar o histórico de suas atividades, visualizando ações registradas e informações de pontuação. O histórico permite acompanhar a própria trajetória dentro do jogo e funciona como um registro das iniciativas sustentáveis realizadas ao longo do tempo.

## 3. REPOSITÓRIOS DO GITHUB E LINK DO DEPLOY EM NUVEM

Back-end em Java:

https://github.com/Jonatanfeitosa/SoulGame-Sprint4-Java

Front-end:

https://github.com/Jonatanfeitosa/SoulGame-Sprint4-FrontEnd

Deploy na Nuvem:

https://soulgame-sprint4-java.onrender.com/api

https://soulgame-sprint4-java.onrender.com

Front End no Vercel:

https://soul-game-sprint4-front-end.vercel.app/

## 4. ARQUITETURA E TECNOLOGIAS

A solução utiliza Java 17, Spring Boot 3.3.5 e Maven para a API REST. A persistência utiliza JDBC, driver Oracle e classes DAO. O código está organizado em pacotes de domínio (model), acesso a dados (dao), regras de aplicação (service), contratos (interfaces), controladores REST (resource), conexão (conexao) e configuração (config). Fluxo de execução: navegador/front-end Vercel → requisições HTTP/JSON → API Spring Boot no Render → classes de acesso a dados → Oracle. O pacote config contém o CorsFilter; o tratamento global de exceções padroniza respostas de erro.

A aplicação foi estruturada em camadas para separar as responsabilidades do sistema. O pacote model representa as entidades de domínio; dao concentra as operações de persistência no Oracle por meio de JDBC; service organiza as regras de negócio; e resource disponibiliza os endpoints REST utilizados pelo front-end. Os pacotes conexao, interfaces e config complementam a arquitetura, incluindo gerenciamento de conexões, contratos de comportamento, configuração de CORS e tratamento de exceções. Essa organização favorece a manutenção, a reutilização de código e a integração entre os componentes.

## 5. TABELA DE ENDPOINTS

Os endpoints da API REST foram implementados utilizando os métodos HTTP GET, POST, PUT e DELETE, com retornos definidos conforme o resultado de cada operação. Os códigos de sucesso incluem 200 (OK), 201 (Created) e 204 (No Content), enquanto as respostas de erro contemplam 400 (Bad Request), 401 (Unauthorized) e 404 (Not Found). Os códigos apresentados na tabela foram conferidos diretamente nas classes Resource da aplicação Java, refletindo os retornos explicitamente implementados nos controladores REST.

| Verbo | URI | Finalidade | Status HTTP esperado |
|---|---|---|---|
| GET | /api/health | Verificar disponibilidade | 200 |
| POST | /api/login | Autenticar usuário | 200/400/401 |
| POST | /api/usuarios | Criar usuário | 201/400 |
| GET | /api/usuarios | Listar usuários | 200 |
| GET | /api/usuarios/{id} | Consultar usuário | 200/404 |
| PUT | /api/usuarios/{id} | Atualizar usuário | 200/400/404 |
| DELETE | /api/usuarios/{id} | Excluir usuário | 204/404 |
| GET | /api/usuarios/ranking | Consultar ranking | 200 |
| PUT | /api/perfil/{usuarioId} | Atualizar perfil | 200/400/404 |
| POST | /api/acoes | Criar ação | 201/400 |
| GET | /api/acoes | Listar ações | 200 |
| GET | /api/acoes/{id} | Consultar ação | 200/404 |
| PUT | /api/acoes/{id} | Atualizar ação | 200/400/404 |
| DELETE | /api/acoes/{id} | Excluir ação | 204/400/404 |
| GET | /api/atividades | Listar atividades | 200 |
| POST | /api/registros | Registrar ação/evidência | 201/400 |
| GET | /api/registros | Listar registros | 200 |
| GET | /api/registros/{id} | Consultar registro | 200/404 |
| PUT | /api/registros/{id} | Atualiza as coordenadas geográficas de um registro existente. | 200/400/404 /500 |
| DELETE | /api/registros/{id} | Exclui registros sem pontuação atribuída; bloqueia a exclusão de registros pontuados. | 204, 404, 409, 500 |
| GET | /api/historico?usuarioId={id} | Histórico por usuário | 200 |
| GET | /api/desafios-agendados | Listar desafios | 200 |

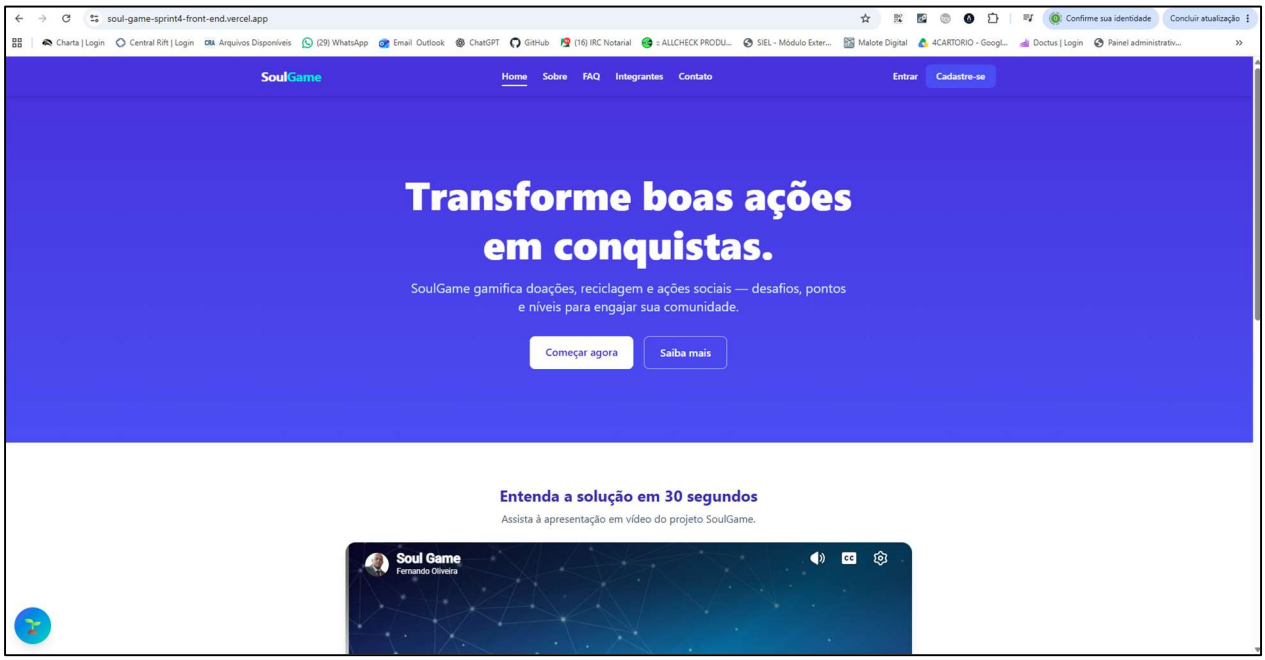
| | | | |
|---|---|---|---|
| POST | /api/desafios-agendados | Agendar desafio | 201/400 |
| DELETE | /api/desafios-agendados | Cancelar desafio | 200 |
| GET | /api/ods | Listar ODS | 200 |
| GET | /api/conquistas?usuarioId={id} | Listar conquistas por usuário | 200 |
| GET | /api/campeonatos | Listar campeonatos | 200 |
| POST | /api/campeonatos | Criar Campeonato | 201/400 |
| POST | /api/campeonatos/inscrever | Inscrever participante | 200/400 |

## 6. PROTÓTIPO FUNCIONAL E INTERFACE DA APLICAÇÃO

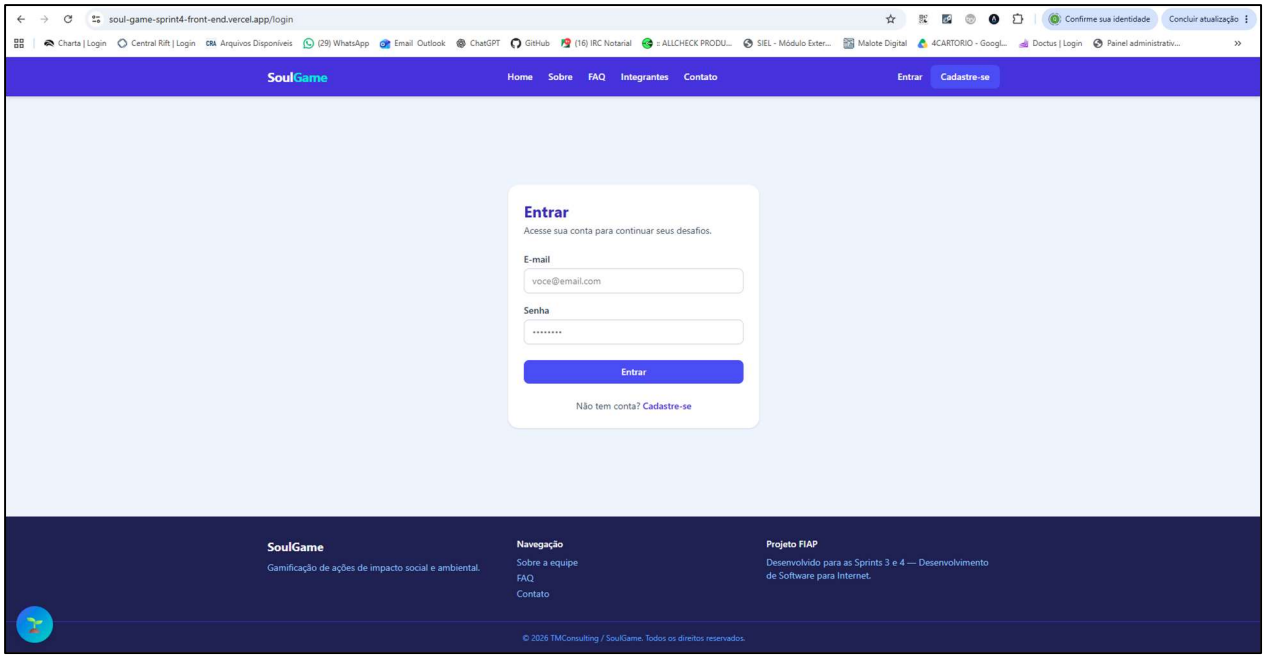
Link do protótipo funcional com deploy no Vercel:

https://soul-game-sprint4-front-end.vercel.app/

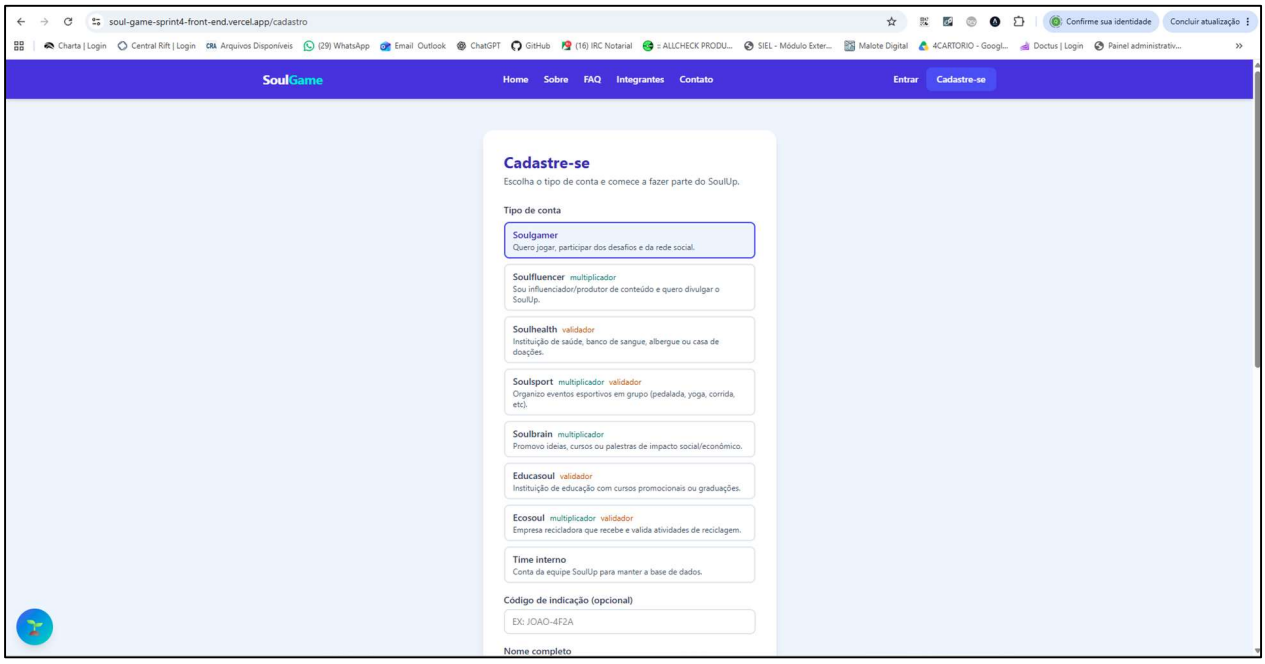
**Tela de início -** demonstra a autenticação do participante por meio da integração entre o front-end e a API Java**:**



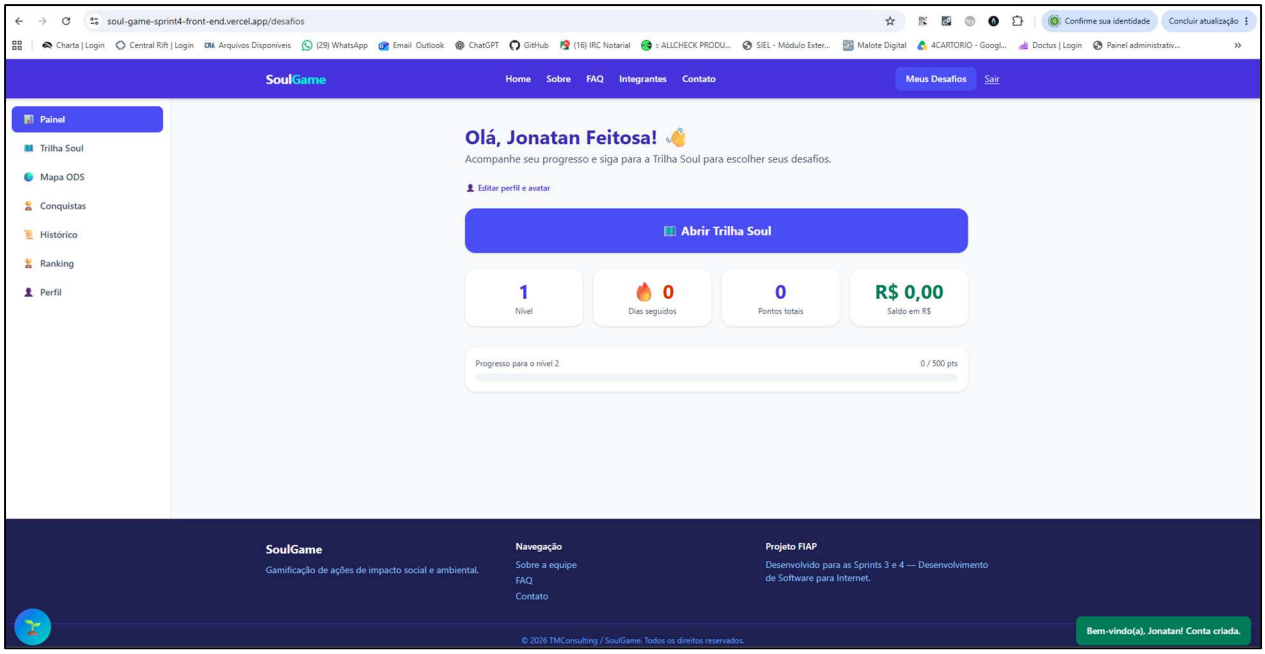

**Tela de Login -** apresentam a criação do usuário e o acesso ao painel individual, com informações de pontuação e evolução**:**



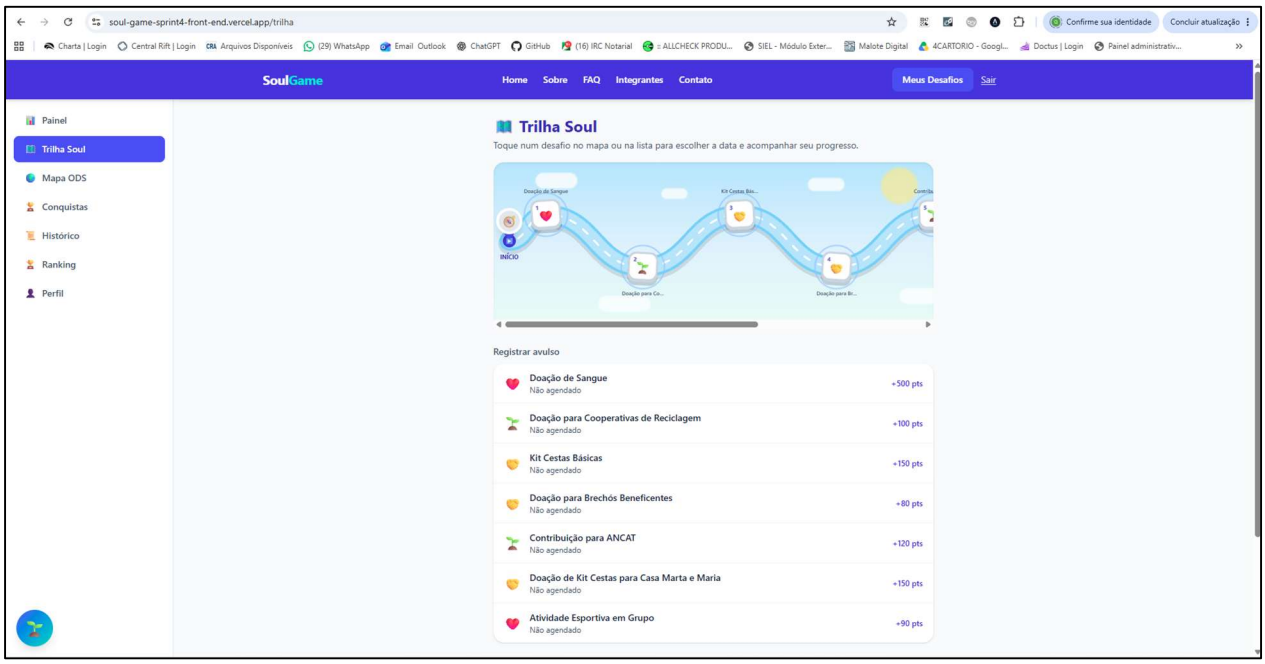

## Tela de Cadastro:

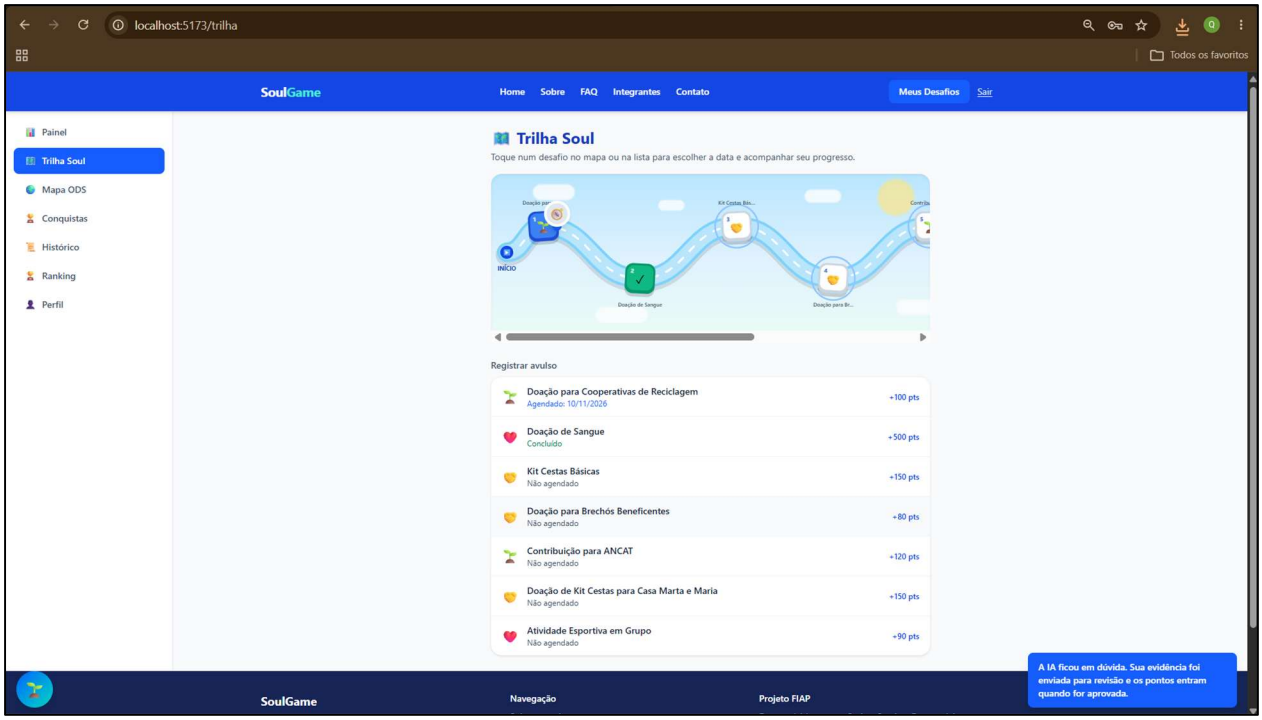



## Cadastro Realizado + Dashboard:

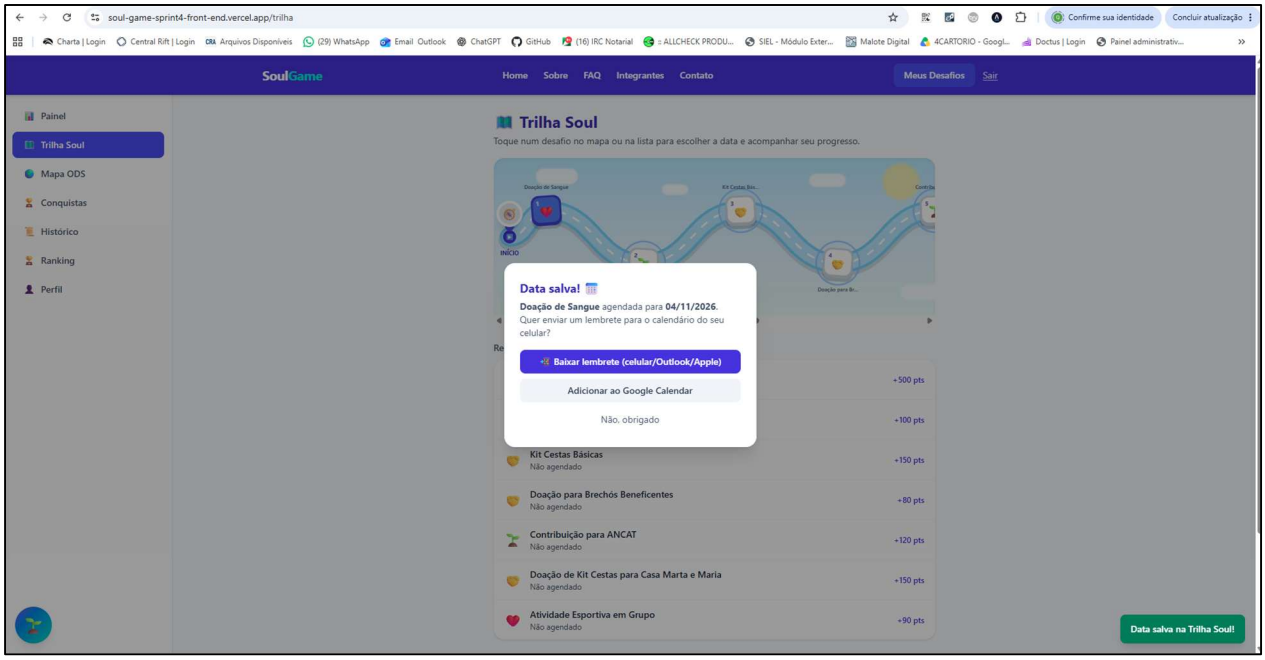



**Trilha Soul -** apresenta as ações sustentáveis disponíveis e a interação do jogador com os desafios:

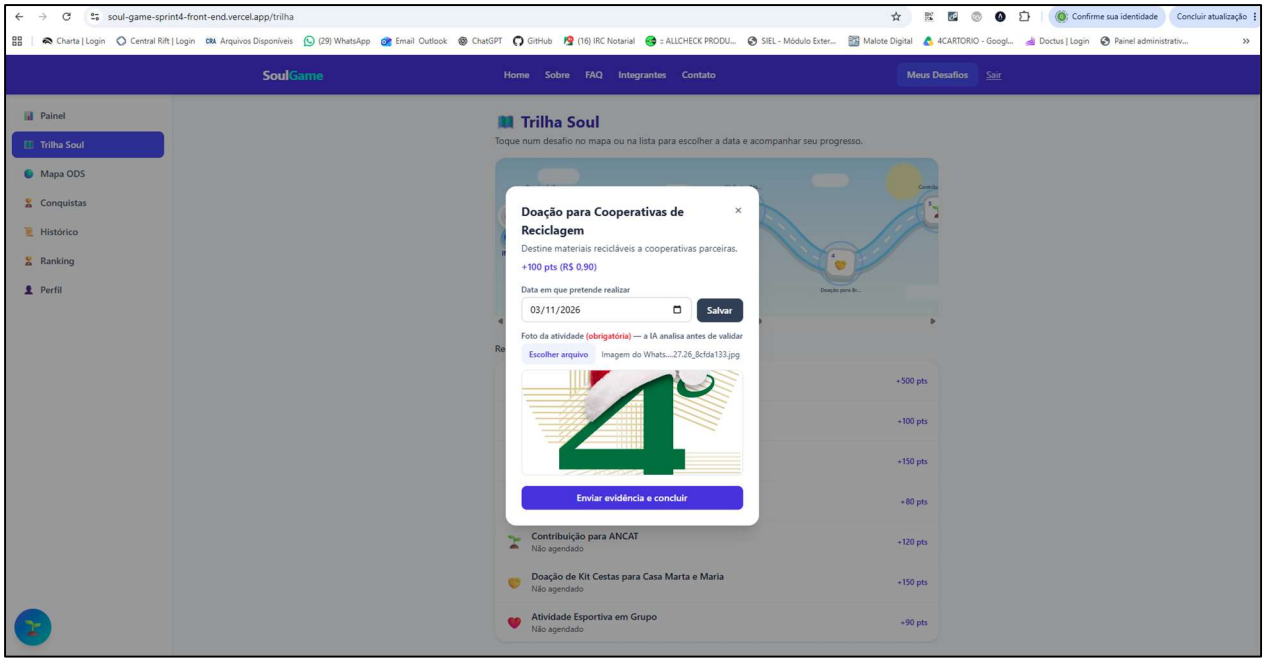


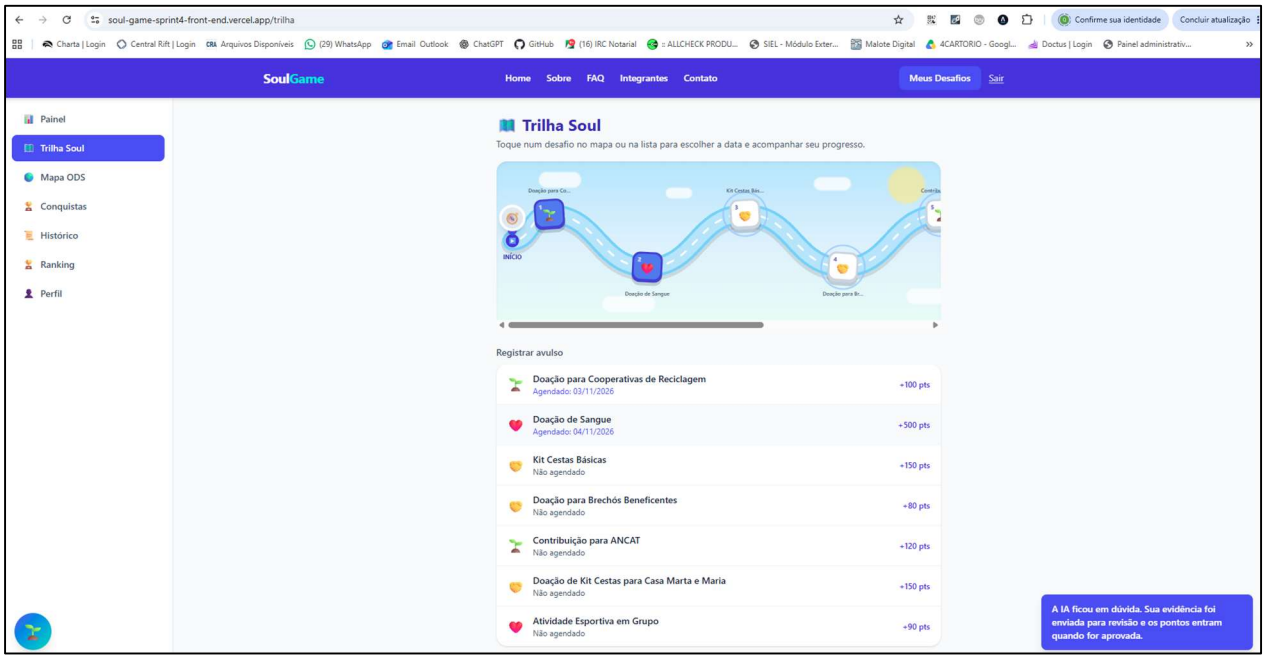



**Agendamento de Atividade** - demonstra a possibilidade de programar desafios para realização posterior:

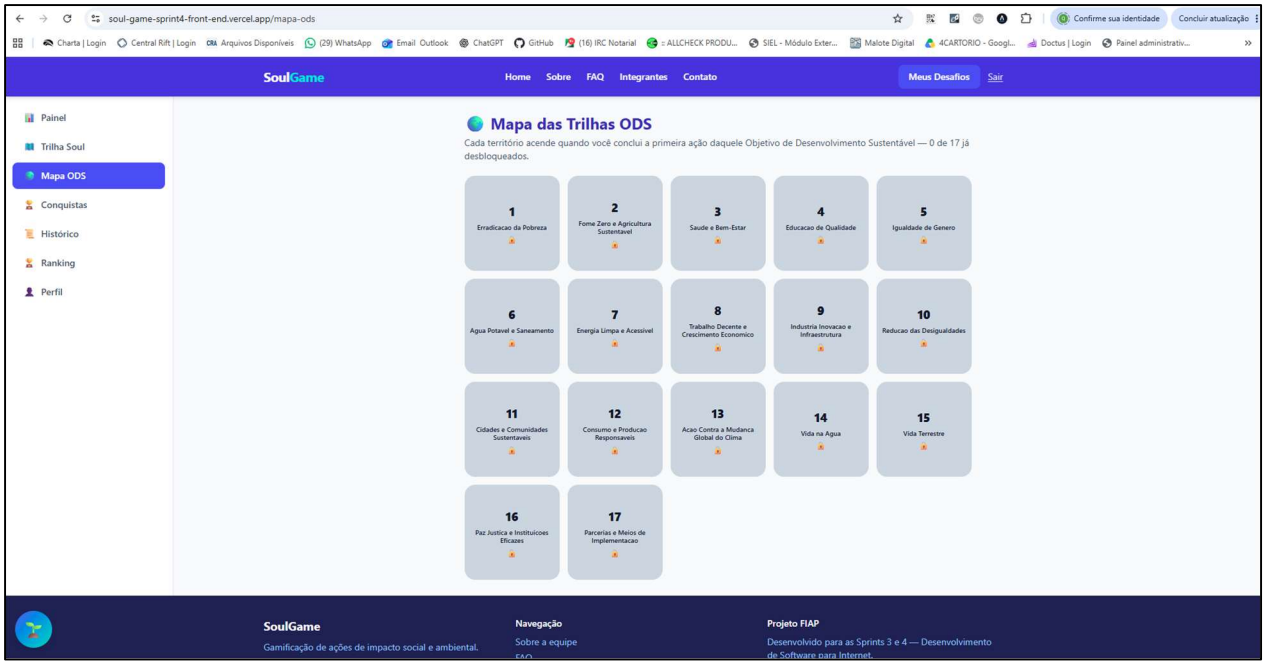



**Envio de Imagem em Desacordo/ Envio de Evidências -** demonstra o registro de uma atividade e o envio de imagem para análise, incluindo o tratamento de evidências inválidas:

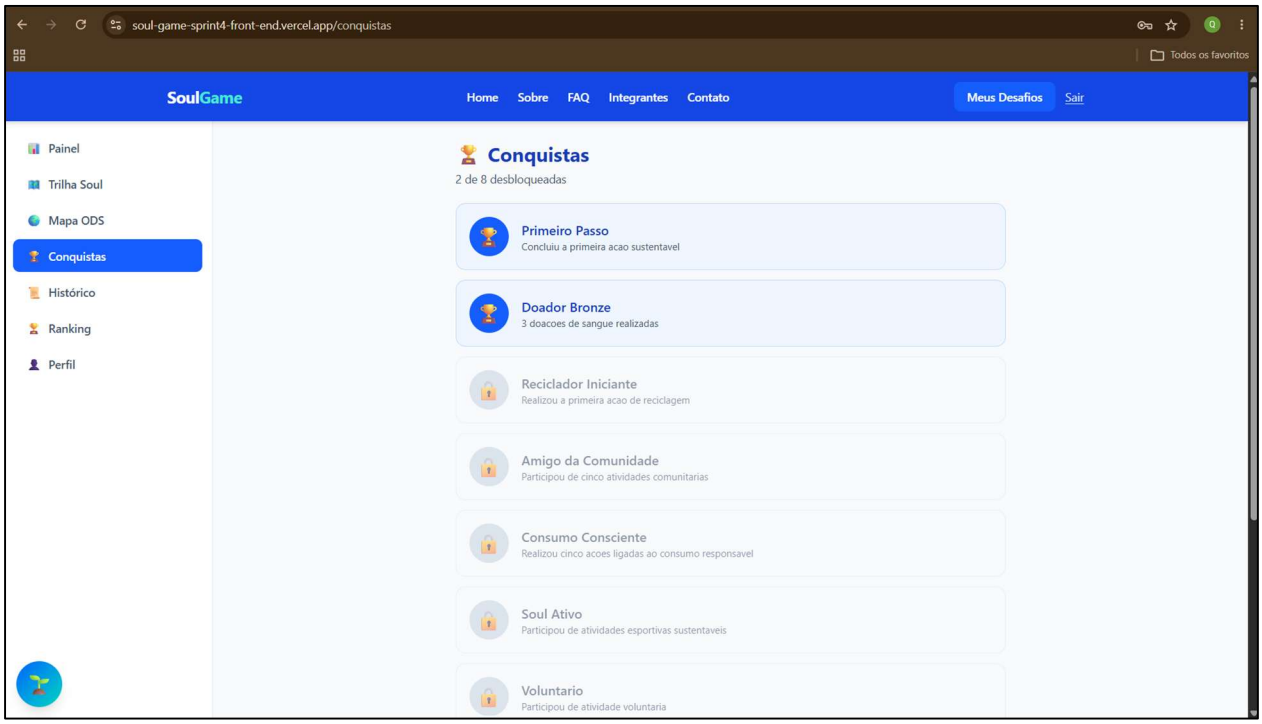


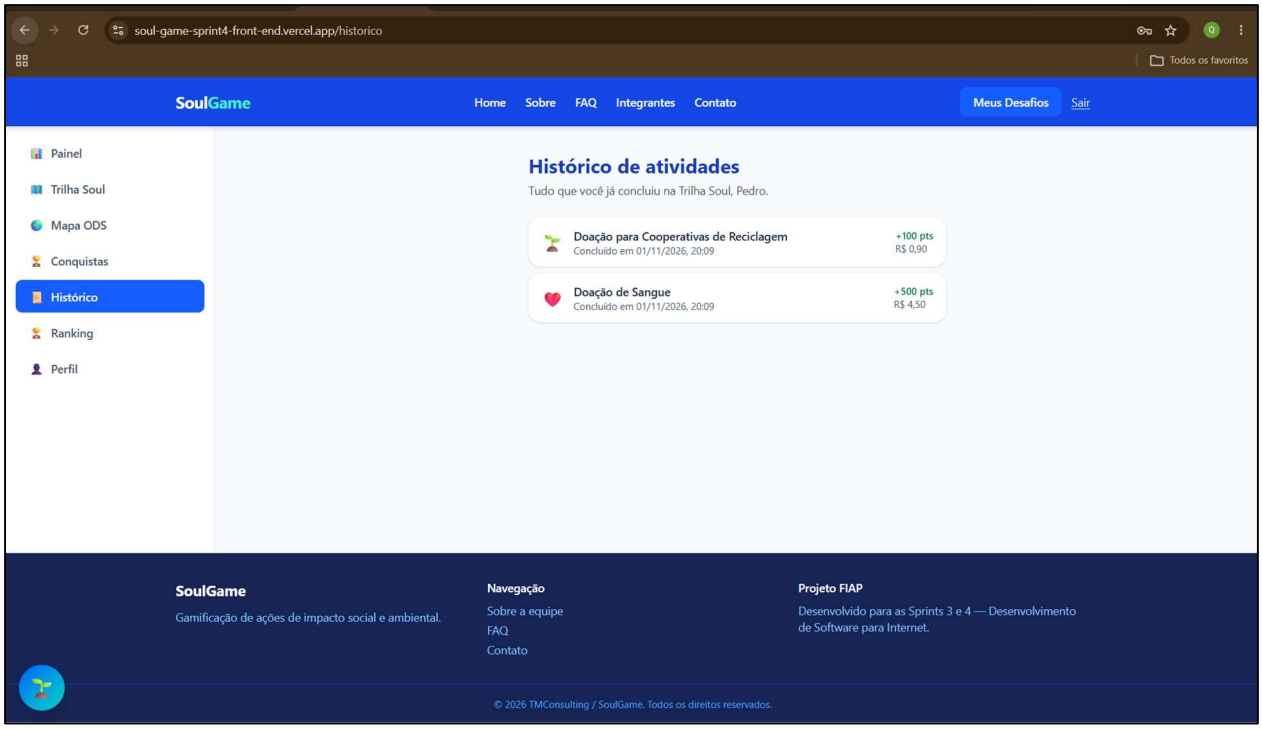

**Mapa das ODS** - apresenta a relação do jogo com os Objetivos de Desenvolvimento Sustentável:



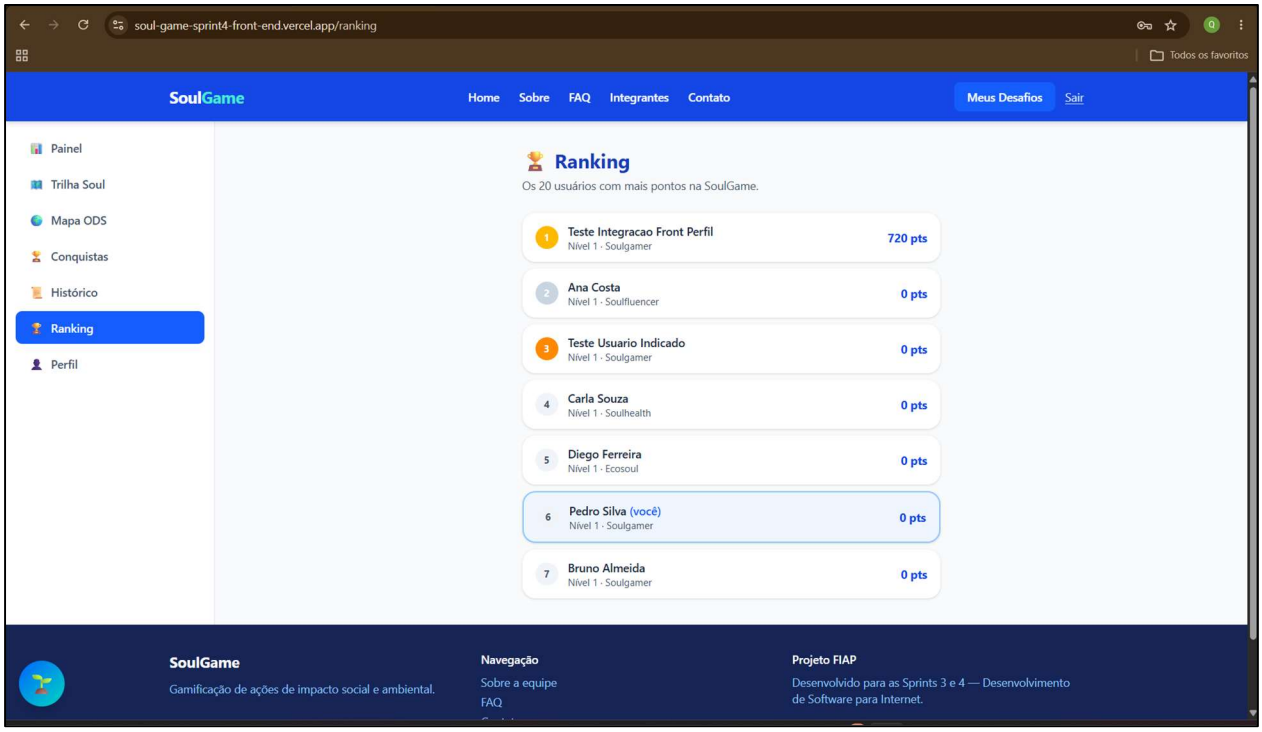

**Conquistas (já coletando informações do Banco de Dados)** - demonstra a consulta das conquistas associadas ao usuário:




**Histórico** - apresenta o acompanhamento das atividades registradas:

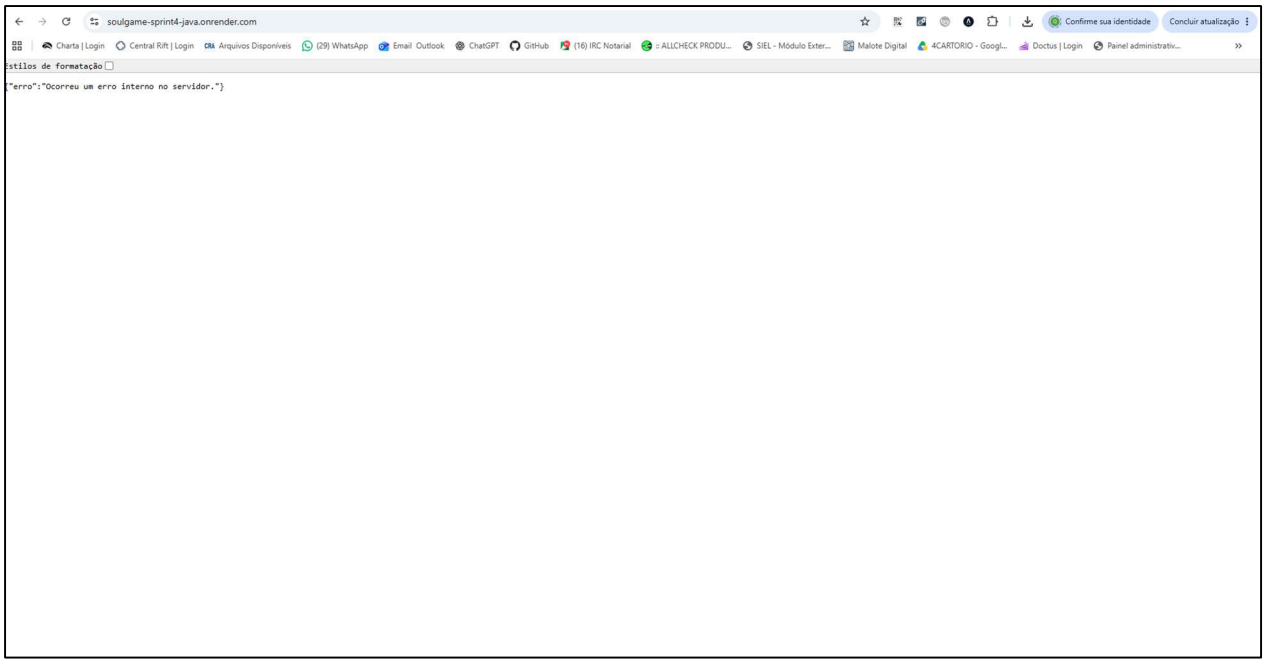



**Ranking** - demonstra a classificação dos participantes conforme a pontuação acumulada:

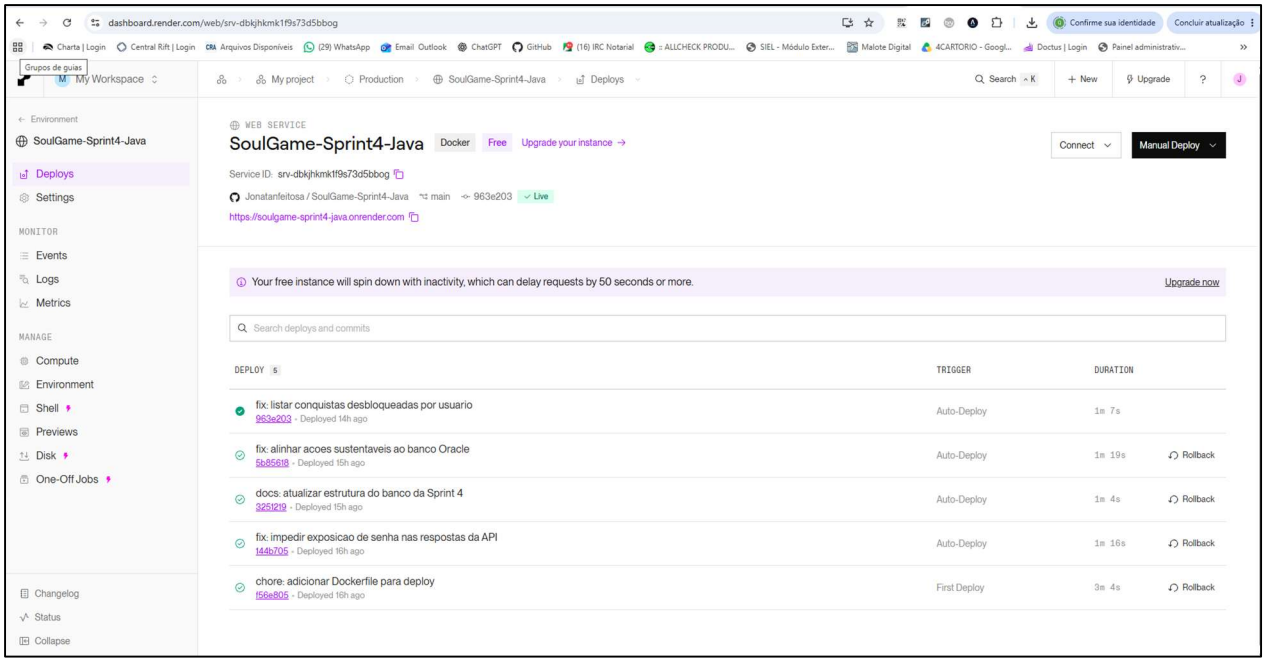



**GlobalExceptionHandler/Validação de dados e tratamento de exceções -** A imagem demonstra o tratamento de uma requisição inválida enviada ao endpoint POST /api/registros, com retorno

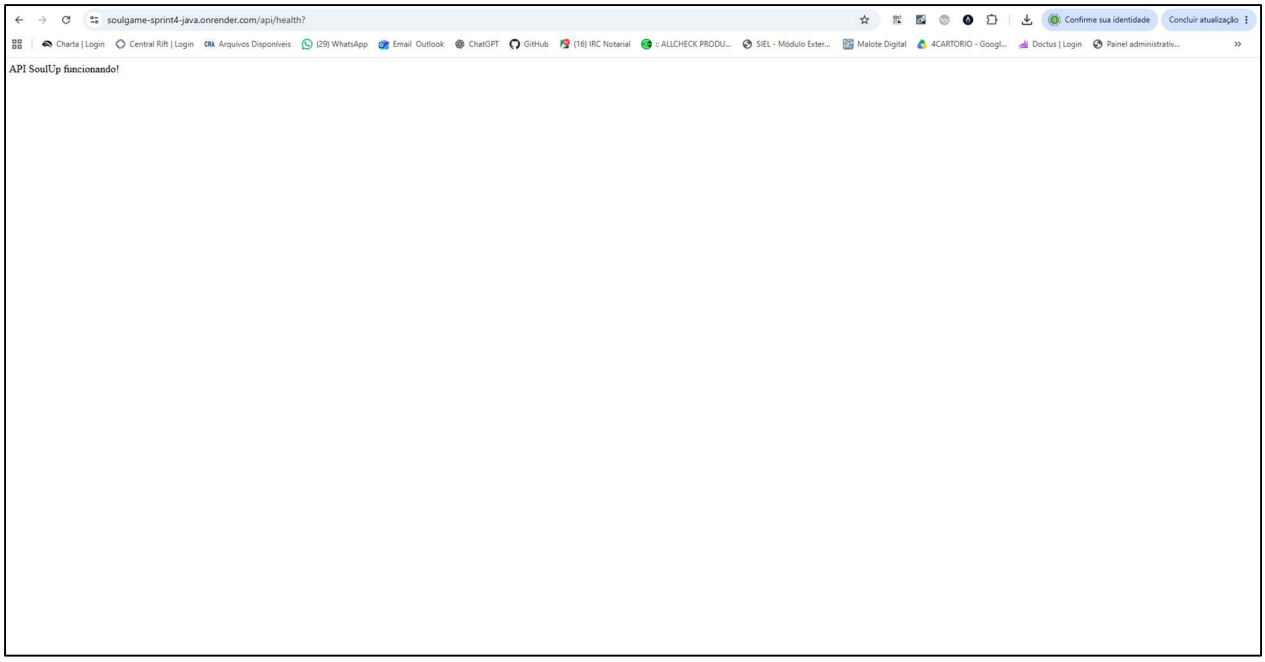
HTTP 400 (Bad Request) e mensagem explicativa em formato JSON. O resultado evidencia a aplicação de validações e o tratamento padronizado de erros no back-end Java.



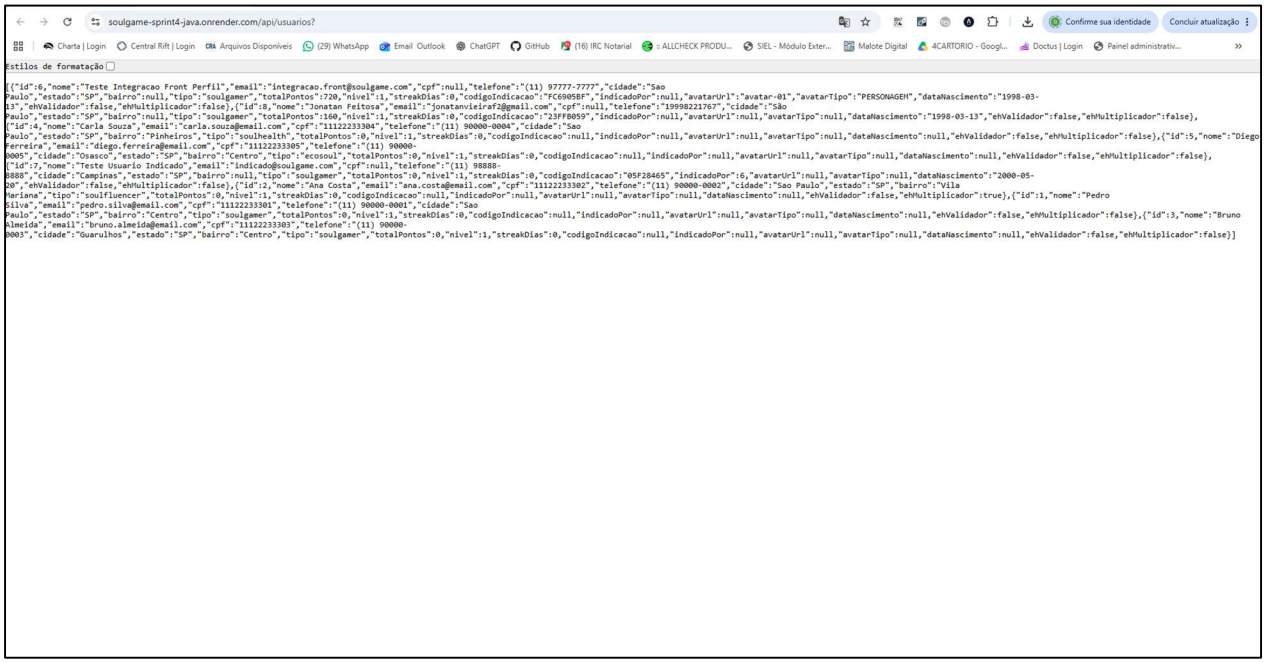
**Implantação do back-end Java no Render** - A imagem apresenta o registro de implantação do SoulGame na plataforma Render, demonstrando a publicação da API REST desenvolvida em Java e Spring Boot em ambiente de nuvem.




**Verificação de disponibilidade da API REST -** A imagem demonstra o funcionamento do back-end Java hospedado no Render, por meio do endpoint /api/health, que retorna a mensagem “API SoulUp funcionando!”, confirmando que o serviço está acessível e respondendo às requisições HTTP.

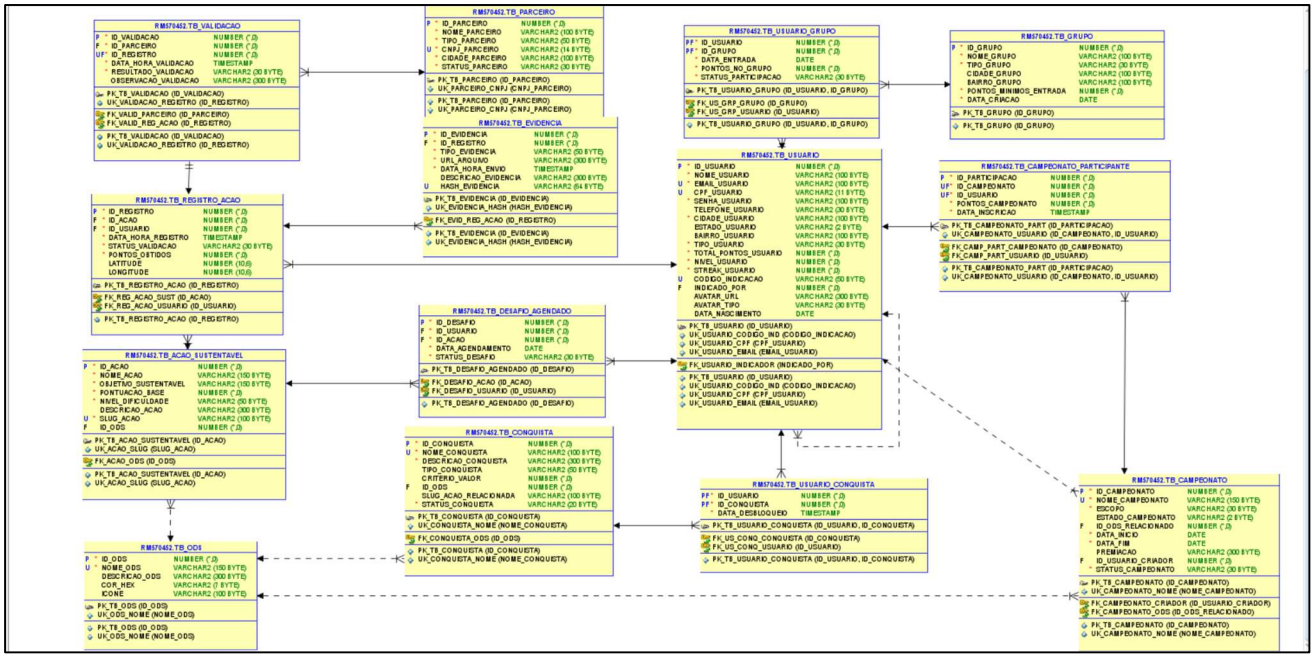



**Consulta de usuários pela API REST** - A imagem demonstra a execução do endpoint GET /api/usuarios, disponibilizado pelo back-end Java hospedado no Render, retornando os registros de usuários armazenados no banco de dados Oracle em formato JSON. A consulta evidencia o funcionamento da camada de persistência e a comunicação entre a API REST e o banco de dados.



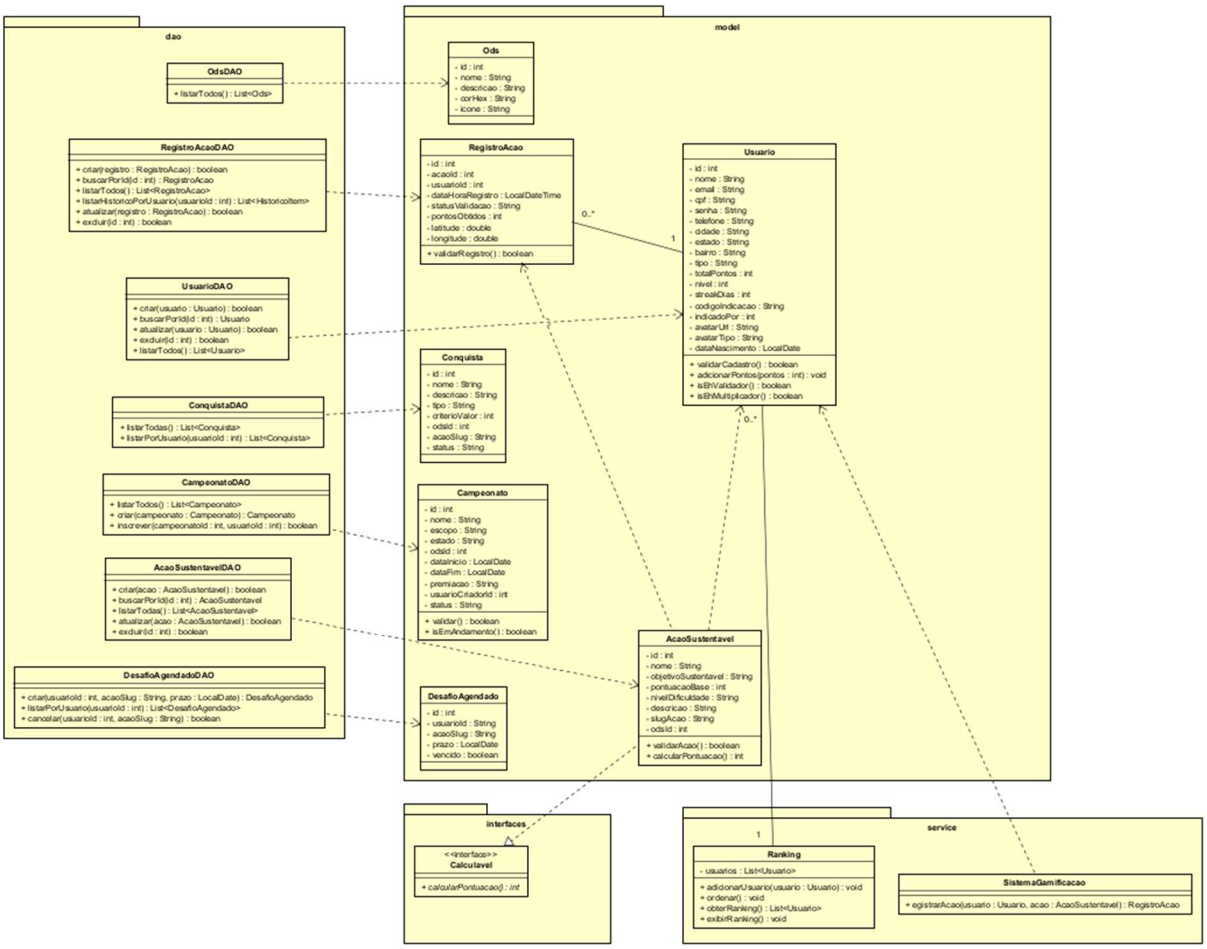

## 7. MODELO DE ENTIDADE-RELACIONAMENTO (MER):

Representa a estrutura relacional do banco de dados Oracle, composta por 14 tabelas responsáveis pelo armazenamento de usuários, ações sustentáveis, registros, evidências, validações, ODS, conquistas, desafios, grupos e campeonatos. O modelo apresenta os relacionamentos e as chaves utilizadas para garantir a associação entre os dados.




## 8. DIAGRAMA DE CLASSES

Apresenta as principais classes da aplicação Java e suas relações, com destaque para as entidades de domínio, classes DAO, serviços e interfaces. O diagrama representa a estrutura central da solução orientada a objetos, enquanto os componentes REST e de configuração complementam a implementação.




## 9. PROCEDIMENTOS PARA RODAR A APLICAÇÃO

Para executar o SoulGame localmente, é necessário possuir **Java JDK 17, IntelliJ IDEA, Maven e acesso ao banco de dados Oracle**. Inicialmente, deve-se baixar o projeto no repositório GitHub, abri-lo no IntelliJ IDEA, configurar o JDK 17 e as credenciais de conexão com o Oracle. Após o carregamento das dependências Maven, basta executar a classe principal `GamificacaoSustentavelApplication`, iniciando a API REST em `http://localhost:8080/api`. O funcionamento pode ser verificado pelo endpoint `/api/health`. O sistema também está disponibilizado em nuvem, com o **front-end hospedado na Vercel** (https://soul-game-sprint4-front-end.vercel.app) e o **back-end hospedado no Render** (https://soulgame-sprint4-java.onrender.com/api), permitindo acessar o jogo diretamente pelo navegador, sem necessidade de instalação local.

**Configuração das variáveis de ambiente**

Para executar a aplicação, é necessário configurar as credenciais de acesso ao banco de dados Oracle por meio das variáveis de ambiente: DB_USER: usuário de acesso ao banco Oracle. DB_PASSWORD: senha de acesso ao banco Oracle.

As variáveis devem ser configuradas no ambiente de execução antes da inicialização da aplicação. No serviço Render, essas configurações são realizadas na seção *Environment*. Por questões de segurança, os valores reais das credenciais não são disponibilizados na documentação nem no repositório público.

## 10. CONCLUSÃO

O desenvolvimento do SoulGame permitiu aplicar os conceitos de orientação a objetos, arquitetura em camadas, persistência de dados e desenvolvimento de APIs REST com Java e Spring Boot. A solução integra funcionalidades de gamificação da sustentabilidade, como desafios, pontuação, ranking, conquistas e histórico de atividades, utilizando o Oracle para armazenamento das informações. A integração com o front-end e a disponibilização dos serviços em nuvem demonstram a aplicação prática dos conhecimentos desenvolvidos ao longo da disciplina, consolidando uma solução funcional e com possibilidades de evolução.

## 11. REFERÊNCIAS

AWORLD. ESG engagement platform for corporate impact. 2026. Disponível em: https://www.aworld.org/act-now/. Acesso em: 17 out. 2026.

CAELUM. **Java e orientação a objetos: curso FJ-11**. [S. l.]: Caelum, [s. d.]. Disponível em: https://www.caelum.com.br/apostilas. Acesso em: 3 nov. 2026.

ECOMATCHER. Corporate tree planting solutions for your business. 2026. Disponível em: https://www.ecomatcher.com/. Acesso em: 17 out. 2026.

ORGANIZAÇÃO DAS NAÇÕES UNIDAS (ONU). Transformando nosso mundo: a Agenda 2030 para o Desenvolvimento Sustentável. 2015.

TETRAMERGE CONSULTING. Challenge TDSPY: documentação da Sprint 3. São Paulo, 2026.